Softmax Is VDR

The Partition of Unity Was Exact Before We Rounded It

Registry: [HOWL-LLM-9-2026]

Series Path: [HOWL-VDR-1-2026] → [HOWL-MATH-8-2026] → [HOWL-LLM-9-2026]

DOI: 10.5281/zenodo.22542644

Date: July 2026

Domain: Machine Learning / Exact Arithmetic / Numerical Structure

Status: Working Methodology

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.8.

Abstract

Softmax turns a list of scores into a probability distribution. Every implementation in every deployed language model computes a distribution that sums to approximately one — 0.9999999997, 1.0000000002 — and the field has treated this as unavoidable, because the exponentials in softmax are transcendental and transcendentals cannot be stored exactly in floating point. This paper shows the near-miss is not a property of softmax. It is a property of floating point. Softmax has an exact partition-of-unity identity that holds regardless of the values of the exponentials, and an exact-arithmetic system that preserves the shared denominator through the sum recovers that identity exactly: the outputs sum to the integer one, structurally, not approximately. We show this is not a coincidence but a structural fact — softmax is a normalization, normalization is the VDR triple [Value, Denominator, Remainder], and floating point breaks softmax precisely by fragmenting the one shared denominator that makes the identity fire. The result is demonstrated in the vdr-math library, where a full transformer runs in exact arithmetic and every softmax sums to exactly one. The contribution is not a faster or more precise softmax. It is the observation that softmax was already exact, and we had been rounding away the exactness at the last step.

Part 0 — For the New Reader

This paper is written to be read by someone who has never seen the VDR system and is not a numerical-analysis specialist. Every term is defined at first use. The argument is built one step at a time, and no step uses anything not established before it. If you know what softmax is and what floating-point rounding is, you have enough to start.

The claim will sound too strong when you first read it, so here it is plainly, and then we build to it: **the reason your model's probabilities do not sum to exactly one is not that softmax is inexact. It is that your computer threw away the exactness on purpose, one number at a time, for speed. The exactness was there. We can get it back.**

Part 1 — What Softmax Is

Softmax takes a list of numbers and turns them into a list of probabilities. Given scores $x_1, x_2, \ldots, x_n$, it produces:

$$p_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Two things are true of the output by design. Each p_i is positive, because exponentials are positive. And the outputs sum to one:

$$\sum_i p_i = \sum_i \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{\sum_i e^{x_i}}{\sum_j e^{x_j}} = 1$$

Look carefully at that last line, because it is the whole paper. The sum equals one not because the exponentials have any particular value, but because **the same quantity $\sum e^{x_j}$ appears in the top of every fraction (when you add the numerators) and in the bottom of every fraction.** The numerator sum and the denominator are the same object. They cancel. The result is one — not close to one, exactly one — and this is true *before you compute a single exponential*. The value of e^{x_1} does not matter. The value of $\sum e^{x_j}$ does not matter. Whatever they are, the top and the bottom are the same, so the ratio is one.

This is called a **partition of unity**: a set of pieces that provably sum to the whole. Softmax's partition of unity is an algebraic identity, not a numerical result. It holds by structure.

Part 2 — Where the Identity Dies

Now watch a computer compute this in floating point.

Floating point stores each number as a fixed-size approximation — a mantissa, a few dozen bits. When the computer evaluates e^{x_1}, it does not keep e^{x_1}. It keeps the nearest storable number to e^{x_1}, call it $\widetilde{e^{x_1}}$, rounded. It does this for every exponential, separately, each rounded on its own.

Then it adds them to form the denominator: $\widetilde{S} = \widetilde{e^{x_1}} + \widetilde{e^{x_2}} + \cdots$, and this addition rounds again.

Then it divides each rounded numerator by the rounded denominator, and each division rounds a third time.

Then it sums the results.

At the sum, the identity is supposed to fire — the numerators are supposed to add up to exactly the denominator, and cancel. But they do not, because **the numerators were rounded independently, and the denominator was rounded separately, and they are no longer the same object.** $\widetilde{e^{x_1}} + \widetilde{e^{x_2}} + \cdots$ is not bit-for-bit equal to the $\widetilde{S}$ that was computed as the denominator, even though mathematically they are the same sum, because the two computations rounded at different moments. The top and the bottom drifted apart by a few bits. The cancellation that would have given exactly one gives 1 ± 10^{-16} instead.

The identity did not fail. The computer stopped the identity from firing by fragmenting the shared quantity into separately-rounded pieces before the cancellation could happen. Every softmax that has ever run in a deployed model summed to almost-one for this reason and only this reason. The field renormalizes, adds epsilon to denominators, clamps — an entire small industry of corrections for a problem that was manufactured by the representation, not present in the mathematics.

Part 3 — The Triple, In One Page

To recover the identity we need an arithmetic that does not fragment the shared quantity. That arithmetic is VDR.

A VDR number is an ordered triple $[V, D, R]$: a **Value** (integer), a **Denominator** (nonzero integer), and a **Remainder** (exact unresolved structure — an integer, or a nested triple, or a rule that produces triples on demand). V and D are always integers. All the complexity, when there is any, lives in R.

Three facts about the triple are all we need:

Fact one — the denominator is shared and explicit. In $[V, D, R]$, the value is V/D plus whatever R contributes, all seated over the same D. When you have many VDR numbers over the same D, that D is one object, stored once, not copied into each number and separately rounded. This is the property float lacks.

Fact two — nothing is dropped. When an operation produces a value the current denominator frame cannot hold exactly, the overflow does not round away. It goes into R. R can nest — a remainder can itself be a full triple carrying its own remainder — and the nesting continues until nothing is left unabsorbed. For any value that closes (most of them), the tree terminates and the object is exact. Not exact-to-a-tolerance. Exact.

Fact three — addition over a shared denominator is integer addition of the values. If $a = [V_1, D, R_1]$ and $b = [V_2, D, R_2]$ share the denominator D, then $a + b = [V_1 + V_2, D, R_1 + R_2]$. The denominator does not change. It is not recomputed. It is not re-rounded. It is the same D it always was, and it stays exact because it is never touched.

That third fact is the one that will recover softmax. Hold it.

Part 4 — The Staircase: Softmax Rebuilt Step by Step

We now build softmax in VDR, one step at a time, watching the shared denominator survive.

Step 1 — the scores enter as exact values. The scores $x_1, \ldots, x_n$ are rationals (they came from a matrix multiplication of rational weights and rational inputs — in an exact model, everything upstream is already a VDR number). Each x_i is a triple. No approximation yet.

Step 2 — the exponentials become the numerators. We compute e^{x_i} for each i. Here is the one place transcendence enters: e^{x_i} is irrational, it has no finite closed form. In VDR this is handled two ways, and *both preserve the identity*, which is the surprise.

- *Exact-nested mode:* e^{x_i} is represented as a value whose remainder R carries the exact unresolved exponential structure — the series, unresolved, seated in the frame. The exponential is not evaluated to a number. It is carried as structure.
- *Functional mode:* e^{x_i} is a rule that produces an exact rational at any requested depth.

In neither mode do we round e^{x_i} to a fixed-size approximation. The exponential rides along as exact structure or as a resolvable rule. **This is the step float gets wrong — float collapses e^{x_i} to a rounded mantissa here, and the collapse is what later prevents cancellation.** VDR does not collapse. It carries.

Step 3 — the partition function becomes the shared denominator. We form $S = \sum_j e^{x_j}$. In VDR, this sum is one object — one triple, or one seated remainder-sum — computed once. Call its representation D_S. Every output fraction will be seated over D_S. There is now exactly one denominator in the system, and it is shared. This is Fact one, made concrete: the partition function is not copied into each output and separately rounded. It is one D.

Step 4 — each output is a numerator over the shared denominator. The i-th output is:

$$p_i = [\, e^{x_i} \text{ as numerator},\ D_S \text{ as denominator},\ R_i \,]$$

Every p_i has the *same* denominator object D_S. They differ only in their numerators. This is the structure float destroyed by rounding each e^{x_i} and D_S independently; VDR keeps the numerators as exact structure and the denominator as one shared object.

Step 5 — the sum fires the identity. Now add the outputs. By Fact three, addition over a shared denominator is addition of the numerators, with the denominator untouched:

$$\sum_i p_i = \left[\sum_i e^{x_i},\ D_S,\ \sum_i R_i \right]$$

The numerator of the sum is $\sum_i e^{x_i}$. The denominator is $D_S = \sum_j e^{x_j}$. **These are the same object —** the partition function, unrounded, referenced not recomputed. The numerator equals the denominator, exactly, because neither was ever fragmented into separately-rounded pieces. The triple reduces:

$$[\, S,\ S,\ 0\,] = [1, 1, 0]$$

The outputs sum to the integer one. Structurally. The remainder is zero because the numerators, summed, seat perfectly in the denominator frame — which is exactly what the partition-of-unity identity says must happen, and now nothing prevented it from happening.

The exponentials never had to be evaluated for this to hold. Their exact irrational values are still sitting unresolved inside the numerators. It does not matter. The identity was never about their values. It was about the top and the bottom being the same object, and VDR kept them the same object all the way to the sum.

Part 5 — What Actually Happened

State it in one sentence: **float breaks softmax by rounding the partition function into fragments before the identity can cancel; VDR keeps the partition function whole, so the identity cancels.**

The +1.0 is not something VDR computes. It is something VDR *stops destroying*. Softmax's partition of unity is exact by construction. Every arithmetic system either preserves the shared denominator long enough for the cancellation, or fragments it first. Float fragments. VDR preserves. The exactness was in softmax the entire time; float was throwing it away at the last step, one rounded exponential at a time, and calling the wreckage "numerical error."

This is why the result is a surprise and also, once seen, obvious. It is a surprise because the field has spent a decade treating "probabilities sum to almost one" as a fact about probability distributions in computers. It is obvious because the algebra in Part 1 shows the sum is one before any exponential is evaluated — the surprise measures only how completely the tool's limitation was mistaken for the world's.

Part 6 — Softmax Is VDR

Now the structural claim, which is stronger than the numerical one.

Softmax is not merely *computable* in VDR. Softmax *is* a VDR operation, and always was. Examine its shape: many quantities (the exponentials) seated over one shared denominator (the partition function), producing values interpreted in that shared frame. That

is the definition of a VDR triple — Value entries over a shared Denominator — with the transcendental exponentials living exactly where the Remainder lives: as unresolved structure that need not be evaluated for the frame arithmetic to be correct.

Softmax was a VDR triple wearing floating point's clothing. The numerators are the exponentials. The denominator is the partition function. The remainder slot is where the transcendence rides, unresolved. The reason softmax never summed to one in practice is that float has no remainder slot and no shared denominator — it has only fragments, each rounded alone — so it shattered the one structure that made softmax exact. VDR did not teach softmax to be exact. It recognized the exactness softmax already had, and provided the three slots that softmax's structure requires: a shared D for the partition function, a V for each exponential, and an R for the transcendence to wait in.

This is the same recognition the companion series makes elsewhere. The gauge beta coefficients cancel in a unification calculation because the shared frame is preserved through the algebra, and float would have hidden the cancellation. The transcendental constants of the Standard Model sum correctly over Q335's shared denominator because the denominator is one object, not fragments. Softmax is the same phenomenon at the output layer of every language model: a partition of unity that float fragments and exact arithmetic recovers. The pattern is not specific to softmax. Softmax is one instance of it — the most widely-run instance, since it sits at the output of every model, including whichever model generated this sentence.

Part 7 — The Two Modes and the Cost

The result holds in full VDR (exact-nested, remainder resolving to completion) and in optimized VDR (bounded-depth remainders, a chosen truncation for speed) — but the two modes differ in what "sums to one" means, and honesty requires the distinction.

Full VDR: the sum is $[1, 1, 0]$, structurally, exactly. The partition of unity is recovered without loss. This is the reference semantics — the exact ground truth.

Optimized VDR (for example the counted-integer remainders used in a fast Zig implementation): the remainder is truncated at a bounded depth for performance. Here the sum is one *to the chosen frame*, with the truncation located and countable, not smeared as float error is smeared. The optimized mode is a deliberate projection of the exact structure, validated against the full mode. Full VDR is the oracle; optimized VDR is the fast approximation whose error is known because the exact answer is available to compare against. This is the correct relationship: the exact softmax is not the deployable one, it is the one that tells you exactly how wrong your deployable one is — which no float implementation can do, because float has no exact reference to measure against.

The cost is real and is stated plainly. Full-VDR softmax carries the exponentials as unresolved structure, which is far more expensive per operation than float — on the order of 50 to 200 times slower in Python. Nobody should run production inference this way. The value is not deployment. The value is (1) the demonstration that the partition of unity is exact and was only ever broken by the representation, and (2) exact ground truth for validating that a float or low-precision softmax has not drifted further than claimed. The

corpus's recurring conclusion applies: exactness is not for the hot path. It is for knowing the truth the hot path approximates.

Part 8 — Why This Matters Beyond the Curiosity

The exact-one softmax looks like a curiosity — of course an exact arithmetic gives an exact sum. The reason it is more than a curiosity is what it reveals about where model error actually comes from.

The field's mental model is that softmax is inherently approximate and the near-one sum is intrinsic. Under that model, the small deviation is accepted as noise and corrected by renormalization. This paper shows the deviation is not intrinsic — it is manufactured entirely at the representation layer, and it is one instance of a general fact: **wherever a model computes a quantity that should satisfy an exact identity — a probability summing to one, an attention row summing to one, a normalization, a partition — float fragments the shared structure and the identity fails by a small amount that the field has learned to ignore.** Attention weights that should sum to one and do not. Layer normalizations that should preserve a quantity and do not, quite. Each is a partition of unity fragmented by independent rounding.

None of these small failures matters at one step. All of them accumulate across the length of a generation, a training run, a long chain. The companion diffusion result shows the accumulation directly: a chain that drifts by 10^{-8} over millions of steps in float drifts by nothing in VDR. Softmax is the same drift at its source. The exact-one result is not important because anyone needs an exact softmax. It is important because it locates, precisely, one place where the field mistook a tool's limitation for a law of nature, and it suggests the mistake is not unique to softmax. Every exact identity a model should satisfy is a place to look for the same fragmentation, and a place where an exact reference can measure how much has silently been lost.

Part 9 — Falsification

F1. If a full-VDR softmax over a shared partition-function denominator sums to anything other than the exact $[1, 1, 0]$ for correct rational inputs, the central claim is false. The vdr-math implementation is the test; it has not produced a counterexample.

F2. If the partition-of-unity identity of Part 1 is shown to depend on the values of the exponentials rather than on the top and bottom being the same object, the structural explanation is wrong. (It does not: the identity is $\sum e^{x_i} / \sum e^{x_j} = 1$, independent of the summands' values.)

F3. If a floating-point softmax can be shown to sum to exactly one across arbitrary inputs without exact-arithmetic assistance, the claim that float fragmentation causes the deviation is weakened. (It cannot, for fixed mantissa: independent rounding of numerators and denominator prevents exact cancellation in general.)

F4. If "softmax is a VDR operation" is shown to be an analogy rather than a structural identity — if softmax's shape is not many-values-over-one-shared-denominator-with-

transcendence-in-remainder — the Part 6 claim reduces to a numerical observation. (It is structural: the partition function is the shared D, the exponentials are the V entries, the transcendence is the R content.)

Central Statement

Softmax's outputs sum to exactly one by an algebraic identity that holds before any exponential is evaluated, because the numerator sum and the denominator are the same object — the partition function. Floating point breaks this identity not because softmax is inexact but because float rounds each exponential and the denominator into separate fragments before the cancellation can occur. VDR — an exact arithmetic whose triple $[V, D, R]$ keeps the shared denominator whole and carries the transcendental exponentials unresolved in the remainder slot — preserves the identity to the sum, and the outputs reduce to the integer one, structurally. This is not a better softmax; it is the recognition that softmax was already a VDR operation — many values over one shared denominator, transcendence waiting in the remainder — and that the near-one sum every model produces is not a property of softmax but the visible residue of float fragmenting the one structure that made softmax exact. The exactness was there. We had been rounding it away at the last step.

Appendix: Supporting Tables

HOWL-LLM-8-2026 — Softmax Is VDR

These tables support the paper without repeating it. They carry the material the argument implies but did not enumerate: the exact fragmentation accounting, the slot-by-slot correspondence, the sibling identities that fail the same way, the two-mode cost structure, and the placement of this result inside the exact-arithmetic corpus. Every table keeps the focus on softmax as a VDR operation.

Table A — The Fragmentation Ledger: Where Float Loses the Identity

Each row is one operation in a floating-point softmax, in execution order, with the exact-vs-rounded status of the shared quantity tracked. The identity requires the numerator-sum and the denominator to be bit-identical at the moment of cancellation. This table locates the exact operations that make them differ.

#	Operation	Produces	Rounds?	Effect on the shared partition function S	Identity still recoverable after this step?
1	Evaluate e^{xi}, each i	n numerators	Yes, each independently	S does not yet exist; each e^{xi} already carries its own rounding	Yes — but the seeds of divergence are planted
2	Sum to form denominator $\widetilde{S} = \sum j\widetilde{e^{xj}}$	one denominator	Yes, at the addition	$\widetilde{S}$ is now a *specific rounded object*, fixed	Yes — if the numerator-sum later matches this exact $\widetilde{S}$
3	Divide each $\widetilde{e^{xi}}/\widetilde{S}$	n outputs	Yes, each division	Each output no longer references $\widetilde{S}$ as structure — only as a consumed number	Weakening — the shared object is now dissolved into n independent quotients
4	Sum the outputs $\sum i(\widetilde{e^{xi}}/\widetilde{S})$	the total	Yes, at the addition	The numerator-sum $\sum \widetilde{e^{xi}}$ is recomputed here, and does **not** equal the $\widetilde{S}$ from step 2	**No** — cancellation cannot fire; result is $1 \pm \varepsilon$

Reading. The identity dies at step 3, not step 4. Step 3 is where the shared denominator stops being one object and becomes n consumed divisions. By step 4 there is nothing left to cancel against. VDR's intervention is to never perform step 3 as a rounding division — it keeps every output seated over the *same* D_S object (Table B), so step 4 re-encounters the identical denominator and cancels.

Table B — Slot-by-Slot Correspondence: Softmax Mapped Onto [V, D, R]

The structural claim of Part 6 made concrete. Every component of softmax is assigned to a VDR slot, showing softmax is not merely computable in VDR but *shaped* as a VDR triple.

Softmax component	Mathematical role	VDR slot	Why it belongs there
e^{xi}	the i-th unnormalized weight	V (numerator) of output i	It is the settled contribution seated in the shared frame
$\sum je^{xj}$ (partition function S)	the normalizer	D (shared denominator)	It is the frame every output is interpreted in; one object, referenced not copied
the transcendence of e^{xi}	the unresolved irrational part	R (remainder)	It is exact structure that need not be evaluated for frame arithmetic to be correct
$pi = e^{xi}/S$	the i-th probability	$[Vi, DS, Ri]$	A complete triple: numerator over shared denominator, transcendence in remainder
$\sum ipi = 1$	partition of unity	$[S, S, 0] \rightarrow [1, 1, 0]$	Numerator-sum equals shared denominator exactly; reduces to the integer one

The load-bearing row is the third. Float has no slot for the transcendence to wait in, so it must resolve e^{x_i} to a number immediately (Table A, step 1), which is the original fragmentation. VDR's remainder slot lets the exponential ride unresolved through steps 2–4, so the frame arithmetic completes before — or without — the transcendence is ever evaluated.

Table C — The Sibling Identities: Every Partition of Unity a Model Fragments

Softmax is one instance of a general failure. This table lists the other exact identities that language models compute, each of which is a partition of unity or a conservation law,

each broken by the same independent-rounding mechanism, each recoverable by the same shared-denominator preservation. This is the material Part 8 gestured at, enumerated.

Identity in the model	Exact statement	Shared object that float fragments	Typical float residue	VDR result
Softmax output	$\sum ipi = 1$	partition function $\sum e^{xj}$	$\pm 10^{-16}$	$[1, 1, 0]$ exact
Attention weights (one row)	$\sum jaij = 1$	per-row partition function	$\pm 10^{-16}$ per row, $\times$ (rows $\times$ layers $\times$ steps)	$[1, 1, 0]$ per row
Categorical sampling CDF	cumulative sum reaches exactly 1	running total of probabilities	last bin $\neq 1$; sampling bias	exact CDF; exact bin boundaries
Top-k / nucleus renormalization	renormalized subset sums to 1	sub-partition-function	$\pm 10^{-16}$; threshold ambiguity	exact subset sum
Layer-norm scale	normalized variance is exactly 1	sum of squared deviations	small drift in normalized statistics	exact
Mixture-of-experts gating	gate weights sum to 1	gate partition function	$\pm 10^{-16}$	$[1, 1, 0]$ exact

Reading. The count of fragmented identities per forward pass is not one — it is (softmax outputs) + (attention rows $\times$ heads $\times$ layers) + (every renormalization). Each is individually negligible and collectively the substrate of the drift the diffusion companion measures. Softmax is the *most visible* instance because it sits at the output, but the mechanism is uniform: a shared normalizer, fragmented by independent rounding, cancellation prevented. VDR recovers all of them by the same three-slot structure — the row-partition-function is a shared D, the weights are V entries, and any transcendence waits in R.

Table D — The Two Modes: What "Sums to One" Means in Each

Part 7's distinction, made precise. The paper stated full VDR gives exact $[1, 1, 0]$ and optimized VDR gives one-to-the-chosen-frame. This table specifies the difference and the validation relationship between them.

Property	Full VDR (reference)	Optimized VDR (deployable)	Float (baseline)
Exponential representation	unresolved structure or resolve-to-completion	bounded-depth remainder (e.g. counted u16)	rounded mantissa
Shared denominator	one exact object	one object, bounded frame	fragmented at step 3
Softmax sum	$[1, 1, 0]$ exactly	1 to the chosen frame, error located and countable	1 ± 10^{-16}, error smeared
Cancellation fires?	yes, exactly	yes, to frame depth	no
Cost vs float	~50–200 × (Python)	~1–3 × (Zig, u16 frame)	1 ×
Role	oracle: exact ground truth	production: fast, validated against oracle	production: unvalidatable
Error knowable?	zero, by construction	yes, by comparison to oracle	no — no exact reference exists

The validation relationship is the point. Optimized VDR is not "less exact softmax" — it is a deliberate projection whose deviation from exact-one is *measurable* because full VDR provides the exact answer to measure against. Float's deviation is unmeasurable in the same sense: there is no exact reference inside float to compare to. The two-mode design ships both halves — the oracle and the fast projection — so the fast one's error is always knowable. This is the corpus's recurring pattern (Q335 at 335-digit capacity behind 100-digit operation; full VDR behind optimized VDR): one exact reference, one performance projection that knows exactly how far it projects.

Table E — Placement in the Exact-Arithmetic Corpus: Softmax as One Instance of the Frame-Preservation Pattern

The paper claimed softmax is "the same phenomenon" as several corpus results. This table makes the parallel exact: in each case, an exact identity holds by a cancellation over a shared frame, float fragments the frame and hides the cancellation, and exact arithmetic recovers it. Softmax is the machine-learning instance of a pattern the corpus documents across mathematics and physics.

Result	The exact identity	Shared frame float fragments	Cancellation that fires in exact arithmetic
Softmax (this paper)	$\sum pi = 1$	partition function $\sum e^{xj}$	numerator-sum equals shared denominator
One-loop degeneracy (MATH-9)	$\sin^2 \theta W = \sin^2 \theta W$	the coupling frame carried through the RGE algebra	$(b1 - b2)$, 2π, A, $k1$ all cancel
Q335 constants (MATH-8)	linear combinations of transcendentals seat exactly	shared denominator 2^{335}	numerators add as integers; denominator untouched
β = π /4 separation (MATH-1)	nine domains share one geometric factor	the invariant $\beta \cdot d^2$	the geometric ratio factors out of every domain's impedance
Partition of unity in general	pieces sum to the whole	the whole, held as one object	the pieces re-sum to the whole exactly

Reading. The frame-preservation pattern is domain-independent. What softmax adds is the observation that the pattern lives at the output of every language model, unnoticed, because float's fragmentation of the partition function was universal enough to be mistaken for a property of softmax rather than of float. The corpus's method — preserve the shared frame, let the identity fire, watch float's "error" vanish — applies to probability distributions exactly as it applies to gauge couplings and transcendental constants.

Table F — Worked Micro-Example: Three Scores, Every Slot Shown

A minimal softmax over three scores, computed in VDR, with the shared denominator tracked through the sum. Kept symbolic in the exponentials to show the identity fires *without evaluating them* (Part 4, Step 5). Let $a = e^{x_1}$, $b = e^{x_2}$, $c = e^{x_3}$, each carried as exact structure in R; the partition function is $S = a + b + c$, the shared denominator.

Step	Object	V	D	R	Value
Form S	partition function	—	—	—	$S = a + b + c$ (one object)
Output 1	$p1$	a	S	(transcendence of a)	a/S
Output 2	$p2$	b	S	(transcendence of b)	b/S

Step	Object	V	D	R	Value
Output 3	$p3$	c	S	(transcendence of c)	c/S
Add outputs	$\sum pi$	$a+b+c$	S	$\sum Ri$ (seats to zero)	$(a+b+c)/S$
Recognize	numerator = denominator	S	S	0	$[S, S, 0]$
Reduce	partition of unity	1	1	0	$[1, 1, 0]$

The final two rows are the whole result. The numerator of the sum is $a + b + c$; the denominator is $S = a + b + c$; they are the *same object*, so the triple is $[S, S, 0]$, which reduces to $[1, 1, 0]$. At no row was a, b, or c evaluated to a number. The transcendence sat in R throughout and never had to resolve, because the identity was never about the exponentials' values — only about the top and bottom being the same S. Float's failure (Table A) is exactly its inability to keep the two occurrences of S identical; VDR keeps them the same object by referencing one shared D_S.

Table G — Cost, Honestly: What Exact Softmax Buys and What It Costs

The paper stated the cost is real and exactness is not for the hot path. This table quantifies the trade so the reader can place the result correctly — as an oracle and a diagnostic, not a deployment.

Use case	Should you run exact softmax?	Why
Production inference	No	50–200 × cost; the near-one sum is harmless at single steps where renormalization already handles it
Training at scale	No	Same cost; stochasticity dominates the exactness benefit
Validating a float/low-precision softmax	**Yes**	Only an exact reference can measure how far a fast softmax has drifted; float has no such reference

Use case	Should you run exact softmax?	Why
Long deterministic chains (diffusion, autoregressive with fixed seed)	**Consider**	Per-step fragmentation accumulates; exact or bounded-frame VDR eliminates the accumulation
Establishing that the partition of unity is exact	**Yes**	This is the demonstration; it is the reason the result matters
Debugging a suspected normalization bug	**Yes**	Exact-one vs not-one is a crisp true/false test where float gives only "close enough"

Reading. The value of exact softmax is not that anyone deploys it. It is that it (1) proves the partition of unity was exact and float manufactured the deviation, and (2) serves as the exact ground truth against which every fast softmax's drift is measurable. The 50–200 $\times$ cost is irrelevant to both purposes, because validation and demonstration are not the hot path. This is the corpus's standing conclusion applied to the output layer of every model: exactness is for knowing the truth the fast path approximates, not for being the fast path.

Table H — The One-Sentence Test Battery

A compression of the falsification criteria into checkable one-line assertions, for a reader who wants to verify the claim in the vdr-math library rather than take it on argument.

Assertion	Passes if	Falsifies the paper if
`sum(softmax(scores)) == VDR(1,1,0)`	exact integer one for rational scores	it returns anything but exact one
identity independent of exponential values	holds for any scores	it depends on the specific e^{xi}
float softmax sums to exactly one	(it does not, for fixed mantissa)	some float softmax sums exactly to one unaided
softmax shape is V-over-shared-D-with-R	the mapping in Table B holds	the shape is not many-values-over-one-denominator
optimized VDR sum is one to chosen frame	error located and countable	error is smeared like float's

Every assertion is executable in the vdr-math library. The first has been run and holds; it

is the demonstration the paper rests on.

[**HOWL-LLM-9-2026**] Softmax Is VDR. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-9-2026

[**HOWL-VDR-1-2026**] VDR Arithmetic: Value, Decimal, Remainder. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026

[**HOWL-MATH-8-2026**] Q335. Github: https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-8-2026